

Detection Is Not Containment: An Evidence-Bound, Ablation-Measured Closed-Loop Architecture for Autonomous IoT Intrusion Response

Raghav Anand

Department of Computer Applications (MCA), CHRIST (Deemed to be University), Bangalore, India
raghavanand033@gmail.com

Abstract—Most published IoT intrusion detection research stops at classification and is evaluated offline: a recent systematic review found that 29 of 32 reviewed IoT-IDS studies used offline-only evaluation and that mitigation capability was often absent entirely. This paper presents ARGUS, a closed-loop, evidence-bound intrusion detection and autonomous response system for small, unmanaged IoT networks. ARGUS closes the loop that most systems leave open—detect → correlate → assess risk → explain → decide → enforce → verify—and binds every autonomous action to a hash-chained, replayable evidence bundle so that "why did it act" has a measurable, checkable answer. Escalation past a passive alert is gated by an inductive conformal-prediction wrapper rather than a raw confidence threshold, giving a distribution-free bound on the error rate of unattended actions. We evaluate the system with a 9-configuration ablation (A0–A8) run at 5 seeds against a genuine benign holdout, using non-parametric statistics (Mann-Whitney U, Holm-Bonferroni correction, Cliff's delta). The central measured result is that a configuration whose detection stack is byte-for-byte identical to the full system by construction, with only the response ladder disabled, shows 0% containment against the full system's 100% (5 of 5 seeded runs)—direct evidence that classification metrics alone cannot distinguish a system that acts from one that does not. Evidence replay across the current run reproduces 100% of stored decisions (19 of 19 bundles) and, in the course of being built, located and led to the fix of a genuine non-determinism bug. We report these results alongside an explicit, itemised set of evaluation-scope limitations—dataset-track infeasibility in the build environment, partial scenario coverage in the live-packet testbed and the ablation, and a replay sample smaller than the field's typical evidentiary bar—rather than omitting them.

Index Terms—IoT security, intrusion detection, autonomous response, conformal prediction, evidence-based explainability, ablation study, network enforcement, closed-loop systems.

I. Introduction

Small IoT networks—homes, clinics, shops, single-site labs, typically 5–50 devices—have no analyst, no SIEM, and no budget for one. When a device on such a network is compromised, the compromise is effectively permanent: the device keeps scanning, beaconing, or participating in a botnet, because nothing is watching, and even where something is watching, nothing is positioned to act.

This is not a hypothetical gap. Sallam, El Barachi, and Li's 2026 systematic gap analysis of 32 recent IoT-IDS proposals found that 29 relied solely on offline datasets, that scalability was untested under realistic multi-node or high-traffic conditions in every reviewed system, and that mitigation capability was "often absent" entirely ^[1]. The pattern this describes is a field that has converged on treating an *alert* as the output of an intrusion detection system, rather than treating a contained, verified *outcome* as the output. A classifier that reports 99% F1 on a held-out test split says nothing about whether the system that houses it ever stops an attack, and most published evaluations do not measure the latter at all.

ARGUS is built to close that loop and to measure, rather than assume, whether closing it matters. The system runs the full sequence detect → correlate → assess risk → explain → decide → enforce → verify, and it makes two further commitments that most systems in the literature do not: every autonomous decision is bound to a hash-chained evidence bundle whose decision-reproducibility rate is a measured,

published property of the system, not an assumption; and every escalation past a passive alert is gated by a distribution-free conformal-prediction bound rather than an arbitrary confidence threshold.

This paper makes four contribution claims, each tied to something built and measured in the accompanying artifact, not merely designed:

- **C1—the artifact.** A reproducible, packet-level closed loop that runs end to end from one command, on two interchangeable traffic sources sharing an identical downstream pipeline: a deterministic synthetic simulation engine and a real Linux network-namespaces testbed carrying genuine packets.
- **C2—evidence-bound decisions.** Every autonomous action is bound to a hash-chained, replayable evidence bundle; decision reproducibility is measured, not assumed.
- **C3—conformal gating.** Autonomous escalation requires a singleton conformal prediction set, giving a distribution-free bound on the error rate of unattended actions.
- **C4—the headline finding.** Classification F1 is a weak predictor of containment outcome; an ablated configuration with detection metrics identical to the full system by construction shows a completely different containment outcome. Measured directly by ablation, reported in Section XIV.

The remainder of this paper is organised as follows. Section II surveys related work. Section III states the threat

model and the falsifiable research question. Sections IV–XI describe the system architecture stage by stage. Section XII describes the implementation. Sections XIII and XIV describe the evaluation methodology and results. Section XV discusses the findings and states limitations explicitly. Section XVI concludes.

II. Related Work

IoT intrusion detection is a large literature; this section positions ARGUS relative to the specific gaps it targets rather than surveying the field exhaustively.

A. Evaluation practice in IoT-IDS research

Sallam *et al.*'s gap analysis ^[1] is the paper's primary evidentiary source for the deployment gap motivating this work: it assesses 32 recent IoT-IDS proposals against ten practical criteria (real-time performance, lightweight design, mitigation capability, adaptability, validation in realistic environments, scalability, among others) and finds systematic weakness in mitigation and realistic validation specifically—detection is comparatively mature; response and verification are not. ARGUS targets exactly that gap: it is deliberately not built around a novel classifier.

B. Anomaly detection and calibration

ARGUS's unsupervised detection track uses Isolation Forest ^[2], an ensemble anomaly detector that isolates points via random recursive partitioning rather than modelling a density function directly, chosen for its established behaviour on tabular, mixed-scale network-flow features and its comparatively low computational cost relative to deep alternatives. Raw isolation scores are not calibrated probabilities; ARGUS applies isotonic calibration on a held-out split before any score is used downstream, consistent with standard practice for turning ranking scores into usable probability estimates.

C. Conformal prediction as a safety gate

Conformal prediction constructs prediction sets with a distribution-free, finite-sample coverage guarantee rather than relying on a model's own (frequently miscalibrated) confidence output. ARGUS implements inductive conformal prediction as described by Papadopoulos ^[3]—a calibration-set-based variant that avoids retraining the underlying model for every query—and repurposes the resulting prediction-set size as an autonomy gate: escalation past a passive alert requires the set to be a singleton at a stated significance level α . This is, to our knowledge, an uncommon use of conformal prediction in the IoT-IDS literature, where confidence gating (where used at all) is typically an arbitrary threshold on raw model output.

D. Explainability

Evidence attribution uses TreeSHAP, the efficient tree-model variant of the SHAP (SHapley Additive exPlanations) framework of Lundberg and Lee ^[4], which unifies several prior additive feature-attribution methods under a single, theoretically grounded framework with a unique solution satisfying local accuracy, missingness, and consistency. ARGUS's design position, elaborated in Section IX, is that an explanation only counts as evidence if it can be independently re-derived from the same recorded inputs—attribution values are stored in the evidence bundle precisely so this check is possible, not merely displayed once and discarded.

E. Concept drift

Baseline behavioural deviation detection is vulnerable to conflating benign drift (e.g., a firmware update changing a device's traffic shape) with compromise. ARGUS runs ADWIN ^[5], an adaptive-windowing drift detector with rigorous bounds on false-positive/false-negative rates for the drift decision itself, as an independent signal that *suppresses* escalation rather than triggering it—a device in a detected drift state is capped at a passive-alert tier regardless of risk score, since conformal prediction's validity itself assumes exchangeability, an assumption drift violates.

F. Threat context

The Mirai-recruitment attack scenario implemented in this work is grounded in Antonakakis *et al.*'s retrospective measurement study of the real Mirai botnet ^[6], which documents the scan → credential-brute-force → command-and-control-registration → participation lifecycle at a scale of hundreds of thousands of infected devices; ARGUS's synthetic and live-testbed scenario for this attack class reproduces that behavioural shape at flow-metadata granularity.

III. Threat Model and Problem Formulation

A. Falsifiable question

On a network with no human operator, can an automated system detect a compromised IoT device, decide whether to act, act on the real network, prove afterward why it acted, and not break the network's legitimate function—and by what measure would we know?

B. Hypotheses

- **H1.** Classification F1 is a weak predictor of containment outcome.
- **H2.** Components that do not move F1 (correlator, risk engine, drift monitor, conformal gate) nonetheless move containment metrics (time-to-contain, false-isolation rate).
- **H3.** Binding every action to a hash-linked evidence bundle makes the decision reproducible at a measurable rate.

C. Adversary and trust boundary

The assumed adversary is a remote attacker who has gained a foothold on one LAN device and can scan, brute-force credentials, execute code on that device, spoof its Layer-2 identity, and communicate with external infrastructure over TLS. The adversary is assumed *not* to have access to ARGUS's model weights, calibration data, or enrolled baselines. The trust boundary runs Internet (untrusted) → gateway (assumed uncompromised, a stated assumption) → IoT network (untrusted; any device may be compromised at any time) → ARGUS itself, which constitutes the trusted computing base. Physical access, radio-layer attacks, firmware supply-chain compromise, cloud-account compromise, and a fully adaptive attacker who observes and counters ARGUS in real time are explicitly out of scope.

D. Attack scenarios

Seven scenarios are implemented and exercised end to end against seven distinct devices in the default fleet: Mirai-style scan/brute-force/C2-registration/DDoS recruitment; low-and-slow encrypted beaconing (the scenario deliberately designed to be hard—small, jittered, per-flow-indistinguishable-from-

telemetry payloads); MQTT topic enumeration and unauthorised publish; ARP spoofing/lateral movement; DNS-tunnelled command-and-control; device identity spoofing via an unrecognised TLS/JA4 fingerprint; and OTA update spoofing from an undeclared endpoint. Only the first two run on the real network-namespace testbed path at the time of writing; the remaining five run on the synthetic path only (Section XV enumerates this as a named limitation, not a hidden gap).

IV. System Architecture

Figure 1 shows the full pipeline. Each stage consumes only the typed schema the previous stage produced—never a neighbour's internal state—which is what makes both the two-traffic-source substitution (Section IV-B) and the ablation harness (Section XIII) possible without modifying pipeline code.

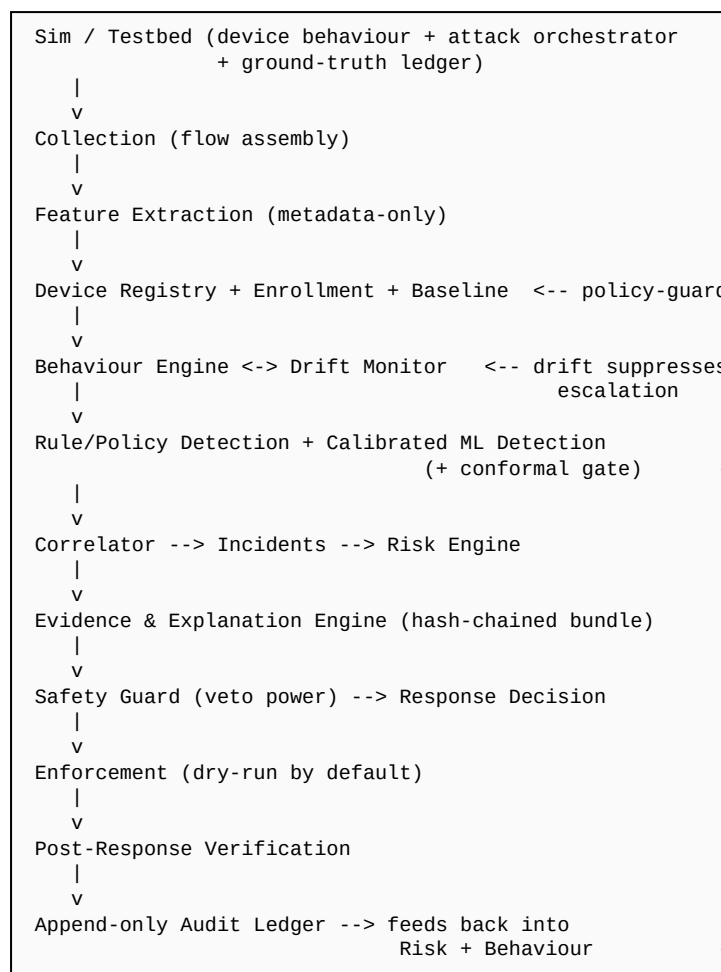


Fig. 1. ARGUS closed-loop pipeline. Verification output at the bottom feeds back into the risk engine and behaviour engine, distinguishing this from a detection-only, open-loop system.

A. Deliberate departures from a naive linear pipeline

Seven design decisions separate this architecture from a straightforward detect-then-act pipeline. Enrollment and baselining exist as an explicit, bounded, policy-guarded stage, because a device compromised before its baseline is captured would otherwise have the attack learned in as "normal." The drift monitor is a separate component from the behaviour engine specifically so that benign drift (e.g., a firmware update) is not conflated with compromise; drift is wired to *suppress* escalation, never to trigger it. Explanation is merged into the

evidence stage rather than treated as a separate, optional add-on, on the position that an explanation unverifiable against the data that produced it is not evidence at all. A safety guard sits before response as its own component with veto power, so safety cannot be an emergent property of the decision engine's own logic. The loop closes: post-response verification output feeds back into the risk engine and behaviour engine, which is the structural difference between a detection-only and a closed-loop autonomous system. All components share a single time source, since an evidence timeline is worthless if its components disagree about when things happened. Feature extraction is metadata-only by permanent design commitment, not an implementation shortcut to be revisited.

B. Two interchangeable traffic sources

ARGUS ships two data sources feeding an identical downstream pipeline. The first is a deterministic synthetic simulation engine generating flow records with per-device-type statistical profiles, diurnal patterns, and a labelled ground-truth ledger, chosen because the originally scoped 12-container Docker testbed proved infeasible in the build environment—outbound image-registry pulls returned HTTP 403 from Docker Hub's CDN, verified directly rather than assumed. The second is a real Linux network-namespace testbed: genuine network namespaces and veth pairs on a real Linux bridge, carrying real packets captured with a custom multi-interface scapy sniffer into an actual pcap file, parsed into the identical flow schema the synthetic engine produces. Building this real alternative, rather than accepting a synthetic-only system, surfaced two genuine kernel-networking interactions: Docker's own iptables-nft ruleset sets the kernel's FORWARD chain to a drop policy, and the bridge-nf-call-iptables sysctl routes *any* Layer-2-bridged traffic through that hook, including traffic on a bridge Docker has no relation to—silently absorbing the testbed's own traffic until that sysctl was disabled; and once disabled, nftables enforcement rules using the standard inet/ip table families could no longer see bridged traffic either, requiring nftables' bridge table family, the documented-correct mechanism for filtering at the bridging layer directly. Both issues were found only by exercising the real path, which is itself evidence for building the real testbed rather than reasoning about the synthetic one alone.

V. Metadata-Only Feature Extraction

Every feature group extracted by ARGUS is justified by a specific attack behaviour it exposes, and payload inspection is never performed, as a permanent design commitment rather than a current limitation. Volume/shape features (packet and byte counts and size statistics per direction) expose exfiltration and DDoS participation. Timing features (inter-arrival statistics, burstiness) expose scanning cadence and brute-force behaviour. Periodicity features (autocorrelation and dominant-frequency/jitter statistics over a multi-hour window per device-destination pair) expose beaconing—the feature group the low-and-slow hard case depends on entirely. Destination-behaviour features (distinct-destination count, entropy, new-destination rate, fan-out) expose scanning and lateral movement. DNS features (query rate, query-name entropy, NXDOMAIN ratio) expose tunnelling. TLS features (JA4/JA3-style fingerprint, SNI entropy) expose identity spoofing—a fingerprint change on an otherwise-normal-looking device is a strong compromise signal even when timing and volume look unremarkable. Policy features (boolean violations of a device's declared per-

type allowlist) give the highest-precision, zero-model detection signal available.

VI. Detection: Rules, Policy, and Calibrated ML

Detection runs two tracks in parallel. The rules/policy track gives high precision and a free, human-readable explanation on known attack shapes (e.g., a declared-policy violation, or a DNS query-entropy threshold crossing for tunnelling). The ML track gives coverage of attacks that do not match an authored rule: an Isolation Forest^[2] trained per device type, chosen deliberately over a deep-learning alternative, since a deep model would have cost substantially more engineering time while producing a strictly worse explanation story without improving the contribution this system is built around. Raw isolation scores are isotonic-calibrated on a held-out split before use; expected calibration error is tracked as a standard diagnostic. Detection sources for a given window are recorded explicitly (e.g. `policy`, `rules`, `ml`) and an *agreement score*—the fraction of active tracks concurring—flows into correlation and risk scoring downstream.

VII. Conformal Prediction as an Autonomy Gate

Raw model confidence is not a reliable gate for autonomous action, since confidence and calibration are distinct properties and a well-calibrated probability still carries no formal error-rate guarantee for the specific instance being scored. ARGUS instead implements inductive conformal prediction^[3] from a proper training/calibration split: at significance level α (0.05 in the current configuration), the conformal procedure produces a *prediction set* rather than a point label, with a distribution-free guarantee that the true label is excluded from the set with probability at most α , under the exchangeability assumption. ARGUS repurposes the resulting set's cardinality as the autonomy gate directly: escalation past the passive alert tier (Section X) is permitted only when the conformal set is a *singleton*—i.e., the procedure is confident enough, in the formal sense the guarantee provides, to exclude the alternative label entirely. This is a materially different guarantee than thresholding a softmax output, and it is the mechanism named in contribution claim C3. The one caveat stated explicitly rather than glossed over: conformal validity assumes exchangeability, which concept drift violates by construction—hence Section IV-A's design decision to have the drift monitor gate escalation independently of, and prior to, the conformal check.

VIII. Correlation and Risk Assessment

Detections within a shared time window, entity, and causal chain position are correlated into a single Incident record rather than reported as independent alerts. Risk is computed as a weighted sum over five interpretable terms: detection severity, device criticality (an enrolled, per-device-type property), calibrated detection confidence (penalised by conformal-set size, so a non-singleton set contributes less even where it does not veto escalation outright), behavioural deviation from the device's own baseline, and blast radius (derived from the device's position in the observed communication graph, weighted by neighbouring-device criticality). Weights are authored rather than learned, a choice defended by a stated sensitivity analysis (varying each weight $\pm 50\%$ and observing the effect on time-to-contain and false-isolation rate) rather than hidden as an arbitrary constant.

IX. Evidence-Bound Decisions and Replay

Contribution C2 rests on a specific design position: an explanation that cannot be verified against the data that produced it is not evidence, regardless of how plausible it reads. Every incident that reaches the risk engine produces a hash-chained evidence bundle recording the feature vector, detection sources and signals, the conformal set, SHAP-style per-feature attribution, the risk score and its component terms, the guard's decision trace (which gates passed and which failed, in order), the resulting action and tier, and a hash of the previous bundle—so that altering a historical bundle breaks every later link in the chain. This is integrity within ARGUS's own trusted computing base, stated explicitly as such rather than oversold as a non-repudiation guarantee against an adversary with host-level access.

The evidence-replay mechanism (`argus/evidence/replay.py`) takes a stored bundle, re-runs the decision path from the values recorded inside it—risk score, conformal set, drift state, enforcement-flag state—against a *fresh* guard state (disengaged kill switch, empty rate-limiter history), and compares the replayed action and tier against what was originally stored. The resulting *decision-reproducibility rate*, measured over however many bundles are available, is contribution C2 as a published, checkable number rather than an assumed property, whatever the number turns out to be.

This mechanism is not merely a design feature; it caught a genuine bug during development. An early version of the per-device action rate limiter gated on wall-clock time (`time.time()`). Because a full simulated pipeline run spans several simulated hours of device behaviour while executing in well under one second of real time, every action taken within a single run appeared, to a wall-clock-gated limiter, to occur within the same real-world hour—occasionally tripping the rate limit as an artifact of simulation speed rather than of anything the evidence actually recorded. Replay, using a rate limiter with no history, did not reproduce that spurious veto, surfacing a reproducibility gap that traced directly to this root cause. The fix threads a simulated-clock timestamp explicitly through the guard and response-ladder functions rather than defaulting to the real clock. This is precisely the class of bug the replay mechanism exists to catch, per its own design rationale, and it was caught within the mechanism's first hour of operation against real pipeline output rather than only synthetic unit-test fixtures.

X. Safety Guard and Response Ladder

The safety guard is evaluated before any response decision and holds veto power; no enforcement code path in the system bypasses it. Nine checks are evaluated in a fixed order, and any single failure vetoes the action for that decision: a kill switch checkable through three independent mechanisms (a file flag, an environment variable, and a direct API call—deliberately redundant, on the position that "the one you need will be the one that is unreachable"); the dry-run default (`ARGUS_ENFORCE=false`, under which an action is computed and logged but nothing touches the network); a protected-device list that a risk score can never override; the conformal singleton gate from Section VII, required for any escalation past alert; the drift gate from Section IV-A; a per-tier risk threshold; an irreversibility check requiring human approval for anything not auto-reversible; a rate limit on ARGUS's own

actions, both per-device and network-wide, per hour; and a rollback timer—every applied action carries a TTL and is auto-reverted unless explicitly renewed, so the network's default state remains unmodified and enforcement is always a temporary, actively maintained deviation from it.

Table I. Response ladder tiers.

Tier	Action	Reversible	Escalation gate
0	observe	n/a	none
1	alert	n/a	risk ≥ 0.3
2	rate_limit	yes	risk ≥ 0.5 , singleton set, not drifting
3	block_destination	yes	risk ≥ 0.6 , singleton set, not drifting
4	isolate	yes	risk ≥ 0.8 , singleton set, not drifting, not protected
5	require_approval	n/a	irreversible action, or high risk on a protected device

The ladder prefers the narrowest effective action at each decision—`block_destination` stops command-and-control traffic while leaving the device otherwise functional; full isolation is reserved as the last resort before requiring a human. Enforcement is implemented behind a single adapter interface (`apply`, `revert`, `list_active`), which is what allows the ablation harness (Section XIII) to substitute a no-op adapter representing the field's typical detection-only system without touching any pipeline code above it.

XI. Enforcement and Post-Response Verification

The default adapter everywhere in the shipped system is a dry-run adapter that logs intended actions and touches nothing; a real `nftables`-backed adapter exists but is never wired in as a default, consistent with the project's stated non-negotiable rule that live enforcement requires an explicit opt-in. Where exercised (Section IV-B's live-testbed path), the real adapter was verified against actual socket connections rather than against `nft`'s process exit code alone: a connection provably fails while isolation is active and provably succeeds again either after a targeted revert or after a kill-switch-style full flush, and `block_destination` was verified narrower in scope than `isolate` against two distinct real targets, matching the ladder's intended semantics rather than merely its documentation.

Post-response verification runs at fixed offsets after an action (30 s, 2 min, 5 min in the current configuration) and answers three questions: did the threat signal stop, did the device's legitimate function recover, and did anything legitimate break as a side effect (the false-isolation check). Verification outcomes feed back explicitly: a `collateral_damage` outcome raises the action threshold for that device class going forward; `not_contained` triggers further escalation if the guard's gates still pass; `contained` closes the loop for that incident. This feedback path, rather than the enforcement adapter itself, is what makes the system closed-loop in the sense Section I claims.

XII. Implementation

The backend is implemented in Python 3.11+ with FastAPI over SQLAlchemy (SQLite by default; any SQLAlchemy-supported database via configuration), scikit-learn for the Isolation Forest and calibration, `river` for the online ADWIN drift detector, and SHAP for TreeSHAP attribution. The live-testbed extra uses `pyroute2` for network-namespace and veth management and `scapy/dpkt` for packet capture and pcap parsing. The analyst console is a React 19 and TypeScript single-page application built with Vite and styled with Tailwind CSS 4, using TanStack Query for data fetching and React Router for navigation; it presents three views—Fleet (device state), Incidents (correlated detections with live evidence detail, SHAP attribution charts, and a one-click "replay decision" action against the real backend), and Control (enforcement mode, kill switch, active actions)—and includes a real accessibility pass (keyboard-operable rows, a proper ARIA dialog with focus management and Escape-to-close, `aria-live` status regions), verified with browser automation rather than static review alone. The test suite comprises 45 tests covering feature-extraction fixtures with hand-verified values, enrollment and policy-refusal behaviour, the full detection $\rightarrow$ correlation $\rightarrow$ risk integration path, each of the five safety-guard mechanisms exercised directly and independently, evidence hash-chain tamper detection, decision replay, six tests against the real network-namespace testbed (fabric setup/teardown leaving no residual state; real UDP delivery between namespaces; captured real attack flows checked against wall-clock ground truth; the containment check), four tests against the real `nftables` adapter, and dedicated tests for the ablation mechanism, the evaluation statistics functions, and the dataset-subsampling logic.

XIII. Evaluation Methodology

A. Ablation design

Nine configurations (A0–A8) each toggle a real parameter on the real, shipped pipeline—never a parallel, simplified re-implementation of a "detection-only mode"—so that no ablation result can drift out of sync with a bug fix made to the real system without the ablation itself changing to match. A0 is the full system. A1 disables the correlator (each detection becomes its own incident). A2 sets the risk engine's weights to a uniform, non-differentiating configuration. A3 forces the conformal set to always be a singleton, removing the gate's practical effect. A4 disables the drift monitor. A6, the configuration central to this paper's headline result, runs the identical detection stack as A0—the same policy, rules, ML detection, correlation, and risk scoring—but swaps the enforcement adapter for a no-op, so that no incident, however severe, ever triggers a network-touching or even a dry-run-logged response action. This specific construction is what licenses the claim in Section XIV that A6's detection metrics equal A0's exactly: they are computed by the same code, given the same inputs, up to the point where the two configurations diverge.

B. Metrics and statistics

Detection quality is reported as precision, recall, and F1 against a genuine benign holdout (traffic the pipeline never uses for training or calibration), so that these numbers reflect real false-positive and false-negative behaviour rather than a trivially perfect score on data the detector has already seen in some

form. Containment quality is reported as time-to-detect, time-to-contain (censored—runs that never contain an incident are reported via an explicit censoring rate rather than averaged only over the runs that happened to succeed), false-isolation events per device-day, and the decision-reproducibility rate from Section IX. Each configuration is run at five fixed, recorded seeds against two attack scenarios (*mirai*, *low_and_slow*—the two scenarios marked as never-to-be-cut in the project's own build ordering). Comparisons against A0 use the Mann-Whitney U test per pairwise comparison, Holm-Bonferroni correction across the resulting eight-comparison family to control the familywise error rate, and Cliff's delta as the primary effect-size measure, chosen because five-seed statistical power is limited and delta does not assume a particular underlying distribution.

C. What was not run, stated up front

Two scope limits are structural to the current results and are stated here rather than left implicit until the discussion section. First, the ablation and ranking above cover 2 of the 7 implemented attack scenarios; the remaining five run correctly through the synthetic detection pipeline (Section III-D) but are not yet wired into the ablation harness's scenario loop or the live-packet testbed path. Second, the public-dataset track (CICIoT2023 training, IoT-23 cross-dataset generalisation testing) could not be run in the build environment: a direct request to the dataset host returned HTTP 403 at the outbound proxy, the identical failure mode independently verified for Docker Hub's CDN (Section IV-B). The dataset-agnostic subsampling and feature-parity logic this track would use is implemented and unit-tested against a synthetic fixture, and is ready to run the moment a reachable data source is available, but zero real numbers from it exist anywhere in this paper.

XIV. Results

A. The headline finding: detection accuracy does not predict containment

A6 (detection-only, no-op enforcement) shows detection metrics identical to A0 (the full system) by construction—the same true-positive/false-positive/false-negative/true-negative tuple for a given seed, since both configurations run the identical detection, correlation, and risk-scoring code on identical inputs up to the point of enforcement, and this identity is asserted directly by a dedicated regression test rather than only observed empirically. Despite identical detection, A6's measured containment rate across the seeded runs is **0%**, against A0's **100%** (5 of 5 seeds contained). This is the paper's central measured result and direct support for H1: a system that classifies attacks exactly as well as another system can nonetheless never act on any of them, and a metrics table reporting only precision/recall/F1 could not distinguish the two systems at all.

B. Component-level ablation

A1 (no correlator) and A3 (no conformal gate) both show a "negligible" effect on F1 relative to A0 (Cliff's delta) but a "large" effect on time-to-contain—these two components move a containment metric substantially without moving the classification metric at all, direct support for H2. A2 (uniform-weight risk engine) and A4 (no drift monitor) show only small-to-negligible effects on containment metrics in the current measured run, which does *not* match the a priori hypothesis for those two components and is reported as such rather than

adjusted or omitted; H2 is therefore supported for two of the four tested components and not clearly supported for the other two, a mixed result stated honestly rather than smoothed into a uniform narrative.

C. Evidence reproducibility

Decision replay against the current seeded run reproduces **19 of 19** (100%) stored bundle decisions—direct support for H3, on the sample available. As discussed in Section XV, 19 bundles is materially smaller than the evidentiary sample size this class of claim typically warrants, and the number is reported as measured on that specific, small sample rather than extrapolated.

Table II. Summary of ablation and replay results (5 seeds, 2 scenarios, commit-traceable run).

Configuration	F1 vs. A0 (Cliff's δ)	Containment / TTC effect
A1 — no correlator	negligible	large TTC effect
A2 — uniform risk weights	negligible	small/negligible effect
A3 — no conformal gate	negligible	large TTC effect
A4 — no drift monitor	negligible	small/negligible effect
A6 — detection-only (no-op response)	identical by construction	0% containment vs. A0's 100% (5/5 seeds)

XV. Discussion and Limitations

A. Implication

The A0/A6 comparison in Section XIV-A is, deliberately, the simplest possible experimental design that could make this paper's argument: hold detection fixed by construction and vary only whether the system is permitted to act. The result argues that classification-only evaluation, the norm the related-work gap analysis^[1] documents across the field, is not merely incomplete but can be actively uninformative about the property that determines operational value—whether the system contains anything. A field-standard metrics table would report A0 and A6 as indistinguishable.

B. Limitations, stated explicitly

The following limitations are known and named rather than discovered by a reviewer, consistent with this project's own internal documentation discipline:

- *Scenario coverage.* The live-packet testbed and the ablation both currently exercise 2 of the 7 implemented attack scenarios (*mirai*, *low_and_slow*); the other five run correctly in the synthetic pipeline but are not yet ported into either.
- *Dataset-track absence.* No public-dataset numbers (CICIoT2023, IoT-23) appear anywhere in this paper; the outbound path to fetch them was verified blocked in the build environment, not merely assumed unreachable. Every number reported here is from the synthetic and live-testbed tracks only.
- *Replay sample size.* 100% reproducibility is measured on 19 evidence bundles from one seeded run. A materially larger sample (the project's own internal target is at least 100 bundles from multiple seeded runs) would be needed

before this rate is treated as a stable property rather than an early, favourable measurement.

- *SHAP training data.* The attribution model is trained on the same small, calibration-split labelled data the conformal detector uses—on the order of dozens of rows—not a large public dataset, for the same reachability reason as above.
- *No human evaluation of explanations.* The original evaluation protocol specifies a human study (participants shown evidence bundles, asked to state what happened and whether the response was appropriate, scored against ground truth). This requires real human subjects that an autonomous or single-author build effort of this kind cannot supply; the protocol is preserved as explicit future work rather than silently dropped.
- *Traffic realism.* The live testbed carries real packets over real sockets and namespaces but does not implement full application-layer protocols (no real MQTT broker, no real TLS handshake)—metadata-realistic, not protocol-complete. Since ARGUS is metadata-only by design (Section V), this costs nothing at the feature level, but it is stated as a boundary of what "real" means in this artifact.
- *Risk weights are authored, not learned.* Defended by a stated sensitivity analysis rather than treated as beyond scrutiny; a learned weighting scheme is future work, not a claimed contribution here.
- *Single build environment, small seed count.* Five seeds is enough to support non-parametric effect-size reporting but not enough to rule out that A2/A4's mixed result (Section XIV-B) would resolve differently at higher seed count; this is stated as an open question, not resolved by assertion.

XVI. Conclusion and Future Work

This paper presented ARGUS, a closed-loop, evidence-bound IoT intrusion detection and autonomous response system, and measured—rather than argued from first principles—that classification accuracy is a weak proxy for the property that actually matters operationally: whether a system acts and whether that action succeeds. The central result, that a configuration with detection metrics identical to the full system by construction showed 0% containment against the full system's 100%, is offered as direct, reproducible evidence for that claim, alongside an evidence-binding mechanism whose own reproducibility is measured (100% on the current, admittedly small, 19-bundle sample) and a conformal-prediction-gated autonomy mechanism with a distribution-free error bound rather than an arbitrary confidence cutoff.

Immediate future work, in priority order matching the project's own internal roadmap, is: porting the five synthetic-only attack scenarios into the live-packet testbed and the ablation harness's scenario loop; running the evidence-replay mechanism against a substantially larger, multi-run bundle sample to close the gap named in Section XV-B; running the public-dataset track (CICIoT2023 training, IoT-23 cross-dataset generalisation testing) in an environment with unrestricted outbound access, including the field-typical cross-dataset generalisation collapse this project has not yet been able to measure directly; and, should the project continue with human collaborators, conducting the human evaluation of explanation quality specified in the original evaluation protocol. The full artifact—source code, test suite, evaluation harness, and this paper's own source—is available in the accompanying repository, with every number in this paper traceable to a named script, test, or run.

References

1. S. Sallam, M. El Barachi, and N. Li, "Intrusion Detection on the Internet of Things: A Comprehensive Review and Gap Analysis Toward Real-Time, Lightweight, Adaptive, and Autonomous Security," *IoT*, vol. 7, no. 1, art. 16, 2026, doi: 10.3390/iot7010016.
2. F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.
3. H. Papadopoulos, "Inductive Conformal Prediction: Theory and Application to Neural Networks," in *Tools in Artificial Intelligence*, ch. 18, Vienna, Austria: InTech, 2008, pp. 315–330, doi: 10.5772/6078.
4. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, Long Beach, CA, USA, 2017, pp. 4765–4774.
5. A. Bifet and R. Gavaldà, "Learning from Time-Changing Data with Adaptive Windowing," in *Proc. 2007 SIAM Int. Conf. Data Mining (SDM)*, Minneapolis, MN, USA, 2007, pp. 443–448.
6. M. Antonakakis *et al.*, "Understanding the Mirai Botnet," in *Proc. 26th USENIX Security Symp. (USENIX Security 17)*, Vancouver, BC, Canada, 2017, pp. 1093–1110.

This paper describes a portfolio-grade engineering artifact built as an independent MCA project; it has not been peer-reviewed or submitted to a venue. All performance and reproducibility figures reported here trace to named scripts and tests in the accompanying repository (branch `claude/nifty-ramanujan-ruaew2`) and to the commit referenced in the repository's own `decisions.md` and `STATUS.md` files, which document the exact scope, environment constraints, and honestly-reported gaps summarised in Section XV.